



Image Classification with Convolutional Neural Networks: A Study on the CIFAR-10 Dataset

Machine Learning





Project Overview

- **Objective:** Build and evaluate a CNN to classify CIFAR-10 images.
- **Focus:** Deep Learning (Track 2) – Convolutional Neural Networks.
- **Dataset:** 60,000 images across 10 object classes.





Dataset Overview



CIFAR-10 dataset (32x32x3 color images).



10 balanced classes:
airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.



Training: 50,000 | Testing: 10,000.

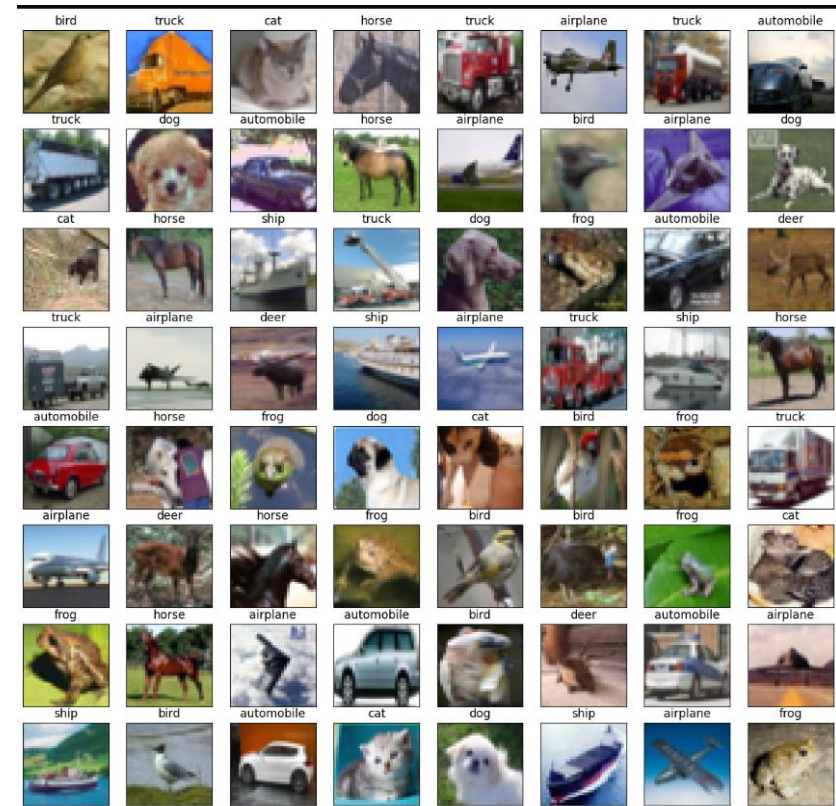


Figure 1: Sample images from each CIFAR-10 class





Data Preparation and Augmentation

- Normalization: training mean & std.
- Split: 45,000 train / 5,000 validation.
- Augmentation: rotation, shift, flip, zoom, brightness, channel shift.





Validation Strategy

- Validation monitors overfitting and tuning.
- Early stopping halts training when loss plateaus.
- Learning rate scheduling improves convergence.

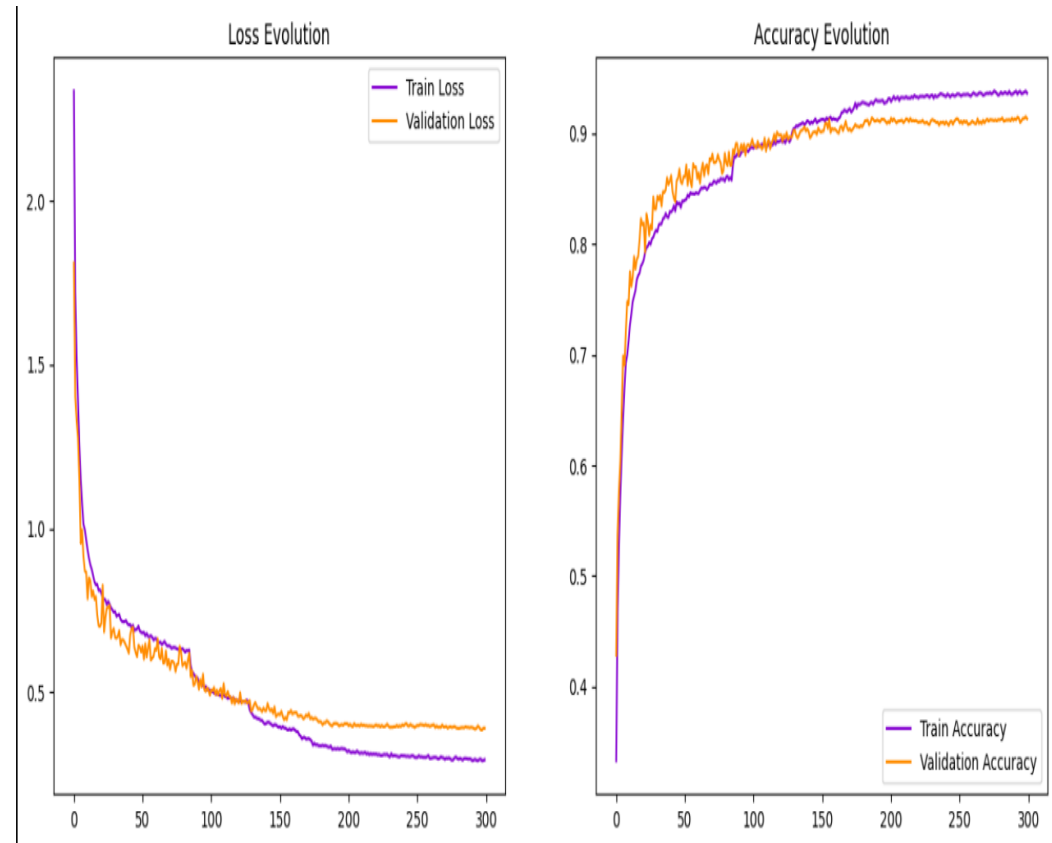


Figure 2: Training and validation accuracy/loss curves





CNN Architecture



4 Convolutional blocks +
Dense output.



Block 1–4: Conv-BN-
MaxPool-Dropout
progression.



Output: Flatten →
Dense(10, Softmax).

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 128)	512
conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
conv2d_6 (Conv2D)	(None, 4, 4, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 4, 4, 256)	1,024
conv2d_7 (Conv2D)	(None, 4, 4, 256)	590,080
batch_normalization_7 (BatchNormalization)	(None, 4, 4, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 2, 2, 256)	0
dropout_3 (Dropout)	(None, 2, 2, 256)	0
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 10)	10,250

Figure 3: CNN architecture diagram showing layers and parameters





Training Strategy

- 300 epochs max with early stopping.
- Batch size: 64 |
Optimizer: Adam
(lr=5e-4).
- Callbacks:
ReduceLROnPlateau,
EarlyStopping,
ModelCheckpoint.

```
Epoch 1/300
704/704 - 79s - 112ms/step - accuracy: 0.3328 - loss: 2.3362 - val_accuracy: 0.4282 - val_loss: 1.8102 - learning_rate: 5.0000e-04
Epoch 2/300
704/704 - 47s - 67ms/step - accuracy: 0.4687 - loss: 1.7236 - val_accuracy: 0.5374 - val_loss: 1.4021 - learning_rate: 5.0000e-04
Epoch 3/300
704/704 - 44s - 63ms/step - accuracy: 0.5302 - loss: 1.5212 - val_accuracy: 0.5768 - val_loss: 1.3299 - learning_rate: 5.0000e-04
Epoch 4/300
704/704 - 46s - 65ms/step - accuracy: 0.5740 - loss: 1.3852 - val_accuracy: 0.6040 - val_loss: 1.2752 - learning_rate: 5.0000e-04
Epoch 5/300
704/704 - 45s - 64ms/step - accuracy: 0.6104 - loss: 1.2540 - val_accuracy: 0.6500 - val_loss: 1.1484 - learning_rate: 5.0000e-04
Epoch 6/300
704/704 - 45s - 64ms/step - accuracy: 0.6435 - loss: 1.1497 - val_accuracy: 0.6988 - val_loss: 0.9542 - learning_rate: 5.0000e-04
Epoch 7/300
704/704 - 46s - 65ms/step - accuracy: 0.6698 - loss: 1.0747 - val_accuracy: 0.6904 - val_loss: 0.9961 - learning_rate: 5.0000e-04
Epoch 8/300
704/704 - 45s - 65ms/step - accuracy: 0.6927 - loss: 1.0124 - val_accuracy: 0.7212 - val_loss: 0.9143 - learning_rate: 5.0000e-04
Epoch 9/300
704/704 - 45s - 64ms/step - accuracy: 0.6998 - loss: 0.9960 - val_accuracy: 0.7480 - val_loss: 0.8695 - learning_rate: 5.0000e-04
Epoch 10/300
704/704 - 46s - 65ms/step - accuracy: 0.7122 - loss: 0.9648 - val_accuracy: 0.7454 - val_loss: 0.8665 - learning_rate: 5.0000e-04
Epoch 11/300
704/704 - 47s - 67ms/step - accuracy: 0.7268 - loss: 0.9343 - val_accuracy: 0.7750 - val_loss: 0.7848 - learning_rate: 5.0000e-04
Epoch 12/300
704/704 - 46s - 66ms/step - accuracy: 0.7365 - loss: 0.9087 - val_accuracy: 0.7616 - val_loss: 0.8491 - learning_rate: 5.0000e-04
Epoch 13/300
...
704/704 - 45s - 64ms/step - accuracy: 0.9383 - loss: 0.2914 - val_accuracy: 0.9152 - val_loss: 0.3897 - learning_rate: 1.0000e-05
Epoch 300/300
704/704 - 44s - 62ms/step - accuracy: 0.9363 - loss: 0.2932 - val_accuracy: 0.9132 - val_loss: 0.3897 - learning_rate: 1.0000e-05
Restoring model weights from the end of the best epoch: 298.
```

Figure 5: Learning rate changes over epochs to show schedule impact





Performance Evaluation

- Test Accuracy: 91.03%
| Test Loss: 0.4118.
- Train: 92.5% |
Validation: 91.1%.

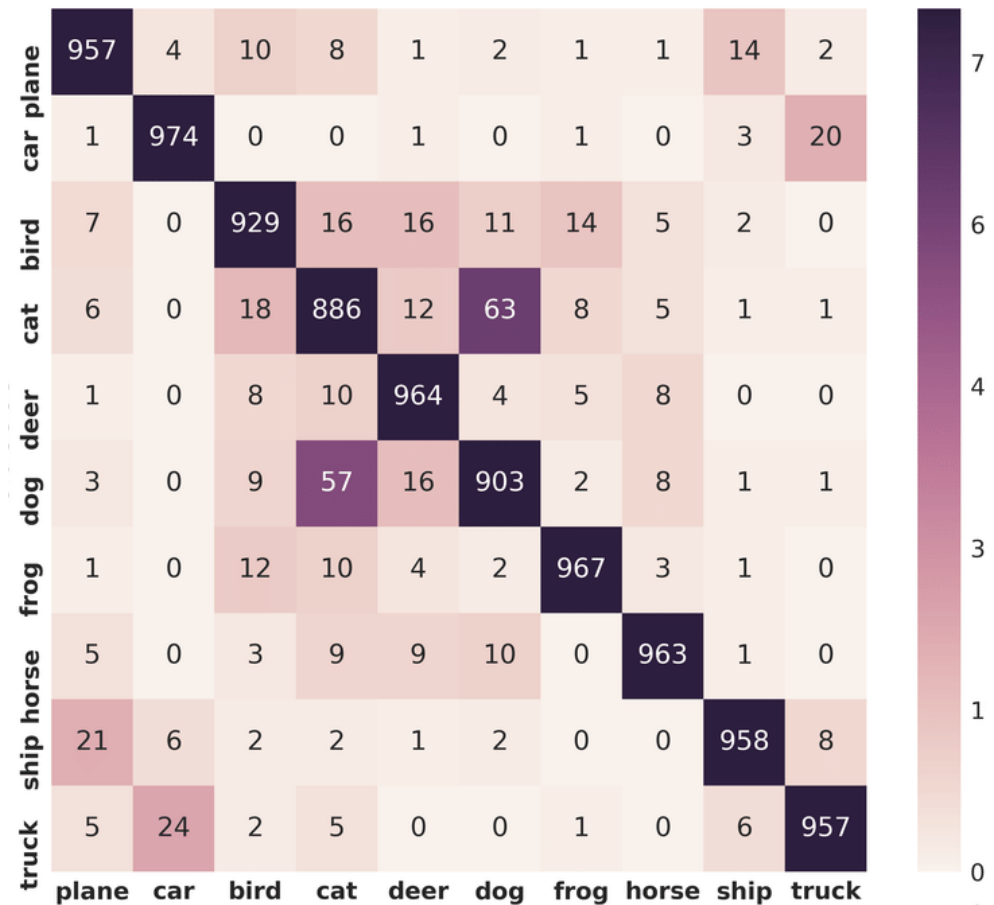


Figure 4: Confusion matrix





Inference Example

- Single image (truck) correctly classified.
- Demonstrates strong generalisation.



```
predicted_class = prediction.argmax()  
  
print('Predicted class: ', class_names[predicted_class])
```

```
Predicted class: truck
```

Figure 7: Prediction





Comparative Discussion

- Classical ML: manual features, limited scalability.
- Deep Learning: automatic feature extraction.
- Advanced ML: high resources, less interpretable.
- **Strengths:**
 - High accuracy and stable convergence
 - Effective use of augmentation and dropout
 - Robust performance on unseen data
- **Limitations:**
 - No per-class F1 analysis
 - Limited interpretability compared to attention-based models
 - Could benefit from transfer learning or deeper architectures





Conclusions & Lessons Learned

- 91% accuracy achieved using CNN on CIFAR-10.
- Regularisation and augmentation improved stability.
- Future work: ResNet, MixUp, transfer learning.





References

- Krizhevsky, A., Nair, V. & Hinton, G. (2009). CIFAR-10 Dataset.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press.
- He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR.
- Ioffe, S. & Szegedy, C. (2015). Batch Normalization. arXiv:1502.03167.
- Kingma, D. & Ba, J. (2015). Adam Optimizer. arXiv:1412.6980.





Q and A

Free discussion

Thank you